Software Engineering II

Workshop 1



CocoaTeaDevs:

Julian Ramirez Diaz

1/10/2026

This document defines the functional and non-functional requirements, the user stories with their acceptance criteria, and the user story map with the project releases.

How the requirements are written

Type	Template used
Functional requirement (FR)	The system must [action / service].
Non-functional requirement (NFR)	The system must [attribute] under [condition] with [metric].
User story (US)	As a [role], I want to [action] so that [benefit]. Acceptance criteria: Given [context], When [event], Then [outcome].

Each requirement expresses one idea, uses a measurable value instead of vague words, and is linked to a way of verifying it. Every functional requirement is linked to a user story.

1. Requirements Documentation

1.1 Stakeholders

Stakeholder	Interest in the system
Players	End users (Bronze to Emerald, LAN server). They want recommendations they can trust, a simple flow and the protection of their data.
Developers (CocoaTeaDevs)	Build and maintain the system, including the Administrator role. They need clear requirements and a system that stays inside the Riot API limits.
Riot Games	Owner of the API and the game data. Requires that the system respects the API rate limits, policies and the disclaimer.

1.2 Functional requirements

ID	Requirement	User story	How to verify
Accounts			
FR-01	The system must allow a player to create an account with a valid email and a password of at least 8 characters.	US-01	Test: register with valid data (account created, redirect to dashboard) and with an invalid email or a short password (rejected).
FR-02	The system must allow a registered player to log in with email and password.	US-01	Test: log in with correct credentials (access) and with wrong credentials (denied).
FR-03	The system must allow a player to delete their account and all associated data.	US-05	Test: delete an account, then check that the profile, Riot profile, playstyle profile and recommendations no longer exist and the login fails.
Riot account and match data			
FR-04	The system must allow a logged-in player to link one Riot ID of the LAN region and save it only after the Riot API confirms it exists.	US-02	Test: link an existing ID (saved and marked as verified) and a non-existent ID (not saved).
FR-05	The system must show an error message that states the cause (ID not found, Riot API unavailable or limit reached) when the Riot API cannot answer a request.	US-02	Test: simulate 404, 5xx and 429 responses from the Riot API and check the message of each one.

ID	Requirement	User story	How to verify
FR-06	The system must fetch the last 20 Ranked Solo/Duo games of the linked account from the Riot API.	US-03	Test: with a mocked account of more than 20 games, check that the request asks for the ranked queue and at most 20 games.
FR-07	The system must exclude from the analysis the games that are remakes or shorter than 5 minutes.	US-03	Test: feed remakes, games of 4 minutes and games of other queues; none of them is counted in the analyzed games.
Analysis and recommendations			
FR-08	The system must derive playstyle tags, each with a score from 0 to 1, from the metrics of the eligible games.	US-03	Test: run a fixed set of games twice; the scores are between 0 and 1 and are the same in both runs.
FR-09	The system must show a warning that the result may not be conclusive, and still show the recommendations, when there are fewer than 10 eligible games or no tag clearly dominates.	US-04	Test: with 9 eligible games the warning appears and the recommendations are shown; with 10 or more games and a dominant tag there is no warning.

ID	Requirement	User story	How to verify
FR-10	The system must show at least 3 recommended champions of the current patch, based on the player's tags.	US-03	Test: the result contains 3 or more champions and all of them belong to the stored patch.
FR-11	The system must show one recommended rune path of the current patch, based on the player's tags.	US-03	Test: the result contains exactly one rune path of the stored patch.
FR-12	The system must show an ordered item build with situational items marked as such.	US-06	Test: the items appear in build order and the situational ones carry the situational mark.
FR-13	The system must recalculate the recommendations every time the player requests an update and replace the previous ones.	US-07	Test: request two updates with different mocked games; the second result replaces the first and its generation date is newer.
Administration			
FR-14	The system must allow an administrator to trigger the update of the static game data (champions, items, runes, images) when a new patch is released, keeping the existing tag mappings.	US-09	Test: trigger the update with a new patch version; the static data and the stored patch version change and the tag mappings remain.
FR-15	The system must allow an administrator to edit tags and tag mappings, and deny that action to players.	US-10	Test: an administrator changes a weight and the next recommendation reflects it; a player trying the same action is denied.
FR-16	The system must show an administrator a dashboard with the Riot API requests per minute versus the limit, the error rate and the requests queued by the rate limiter, refreshed at least every minute.	US-08	Test: open the dashboard, generate requests and check that the three values change within 60 seconds.
FR-17	The system must show a warning on the dashboard when the Riot API usage reaches 80% of the limit.	US-08	Test: set the usage to 80% of the limit and check that the warning appears.
Rate limits			
FR-18	The system must queue or delay the request and show a "try again in X seconds" message when the Riot API usage reaches 80% of the limit (first version of the optimization protocol).	US-03	Test: simulate 80% usage and request an analysis; the request is delayed or queued and the message is shown.

1.3 Non-functional requirements

ID	Category	Requirement	How to verify
NFR-01	Performance	The system must load the main pages (login, dashboard, settings, recommendations) under a standard broadband connection with a load time below 3 seconds.	Measure the deployed pages with browser developer tools (3 runs, all below 3 s).
NFR-02	Performance	The system must show the recommendations under normal Riot API usage (below 80% of the limit) within 20 seconds after the player requests them.	Time 10 analyses of 20 games each; the longest is 20 s or less.
NFR-03	Usability	The system must show a different tip or advice message, taken from a predefined pool, every 5 seconds while an analysis is in progress.	Observe the screen during an analysis; the message changes every 5 seconds.
NFR-04	Security	The system must store passwords only as hashes (managed by Supabase Auth), never in plain text, under any condition.	Inspect the database and the logs: 0 plain-text passwords.
NFR-05	Security	The system must keep the Riot API key and the Supabase service key only on the server and apply Row Level Security to 100% of the tables, so that a player reads only their own data.	Search the browser code for the keys (0 matches); try to read another user's rows (denied).
NFR-06	Compliance	The system must keep the Riot API requests below the Riot rate limits, with the optimization protocol activating at 80% of the limit.	Load test: usage never exceeds 100% of the limit and the protocol starts at 80%.
NFR-07	Compliance	The system must display Riot's required disclaimer (not endorsed by Riot Games) on 100% of its pages.	Inspect every page.
NFR-08	Data freshness	The system must update the static game data to the current patch within 48 hours after a patch release.	Compare the stored patch version with the Data Dragon version within 48 hours of a release.
NFR-09	Availability	The system must keep its Vercel deployment available more than 95% of the time in each month.	Check the monthly uptime report of the deployment.
NFR-10	Usability	The system must let a player link a Riot ID in at most 4 steps: open the app, click "Link Riot ID", enter the Riot ID (GameName#TAG) and region, and confirm.	A tester performs the flow and counts the steps.
NFR-11	Compatibility	The system must work on desktop and mobile devices with Chromium-based browsers (for example Chrome and Edge) using a responsive layout.	Test every page in a desktop window and a mobile viewport in both browsers.
NFR-12	Privacy	The system must store only the login credentials (in Supabase Auth), the Riot ID, PUUID, region, plan and the latest profile and recommendation, keeping 0 raw match records, because its serverless functions keep no data between requests.	Inspect the database after an analysis: there are no match tables or match rows.
NFR-13	Language	The system must show all interface text, messages and tips in one language, English, on 100% of its pages.	Review every page and message; none is in another language.

1.4 Items to be defined

The following items are not defined yet. They are left out of the requirements above so that no requirement stays ambiguous, and the focus of the first releases is the Free version of the app.

Item	Status	What is missing
Premium plan features	To be defined	The plan exists in the data model (Free / Premium) but its features have no requirements yet.
Deeper analysis of the matches	To be defined	Which extra metrics and which extra breakdown a Premium player receives.
Payment or subscription	To be defined	How a player becomes Premium. Out of scope for now.

2. User Stories

There are ten stories for two roles: Player and Administrator. Premium stories will be added when the Premium plan is defined.

US-01: Create account and log in
As a **Player**, I want to create an account and log in so that my preferences and my linked Riot ID are saved and I can come back to them.
Acceptance criteria 1
Given I am on the registration page, **When** I enter a valid email and a password of at least 8 characters, **Then** my account is created and I am redirected to the dashboard.
Acceptance criteria 2
Given I am on the registration page, **When** I enter an invalid email or a password shorter than 8 characters, **Then** no account is created and I see an error message that says what is wrong.
Acceptance criteria 3
Given I have an account and I am on the login page, **When** I enter my email and password correctly, **Then** I am logged in and redirected to the dashboard.

US-02: Link Riot account
As a **Player**, I want to link my Riot ID so that the system can fetch my real Ranked Solo/Duo games.
Acceptance criteria 1
Given I am logged in and on the settings page after clicking "Link Riot ID", **When** I enter my valid Riot ID (GameName#TAG), select the LAN region and confirm, **Then** the system verifies the ID with the Riot API and saves it to my profile, in at most 4 steps counted from opening the app.
Acceptance criteria 2
Given I am linking my Riot ID, **When** the Riot ID does not exist or the Riot API fails, **Then** nothing is saved and I see a message with the cause.

US-03: View champion and rune recommendations
As a **Player**, I want to see recommended champions and a rune path based on my last games so that I can decide who to play next and how to set up my runes.
Acceptance criteria 1
Given my Riot account is linked, **When** I open the "Recommendations" page, **Then** the system analyzes my last 20 eligible Ranked Solo/Duo games and shows at least 3 champions and one rune path of the current patch within 20 seconds.
Acceptance criteria 2
Given the analysis is in progress, **When** I wait for the result, **Then** I see a different tip message from a predefined pool every 5 seconds.
Acceptance criteria 3
Given the Riot API usage is at 80% of the limit, **When** I open the "Recommendations" page, **Then** I see a "try again in X seconds" message instead of the result.

US-04: See a warning when the result may be inaccurate
As a **Player**, I want to be warned when my result may not be conclusive so that I know how much to trust it while I can still see it.
Acceptance criteria 1
Given fewer than 10 eligible games were found, **When** the recommendations are generated, **Then** they are shown together with the warning "may not be conclusive".
Acceptance criteria 2
Given no tag clearly dominates my playstyle, **When** the recommendations are generated, **Then** they are shown together with the same warning.
Acceptance criteria 3
Given 10 or more eligible games were found and one tag clearly dominates, **When** the recommendations are generated, **Then** they are shown without a warning.

US-05: Delete account
As a **Player**, I want to delete my account and all my data so that no personal data of mine remains in the system.
Acceptance criteria 1
Given I am logged in and on the settings page, **When** I confirm the deletion of my account, **Then** my account, Riot ID, playstyle profile and recommendations are removed and I am logged out.
Acceptance criteria 2
Given my account was deleted, **When** I try to log in with my old credentials, **Then** the login fails.

US-06: View the item build
As a **Player**, I want to see an ordered item build with situational items so that I know what to buy during my games.
Acceptance criteria 1
Given my recommendations are available, **When** I open the item section, **Then** I see the items in build order and the situational items marked as situational, all of the current patch.

US-07: Update recommendations
As a **Player**, I want to update my recommendations after playing new games so that they reflect my latest performance.
Acceptance criteria 1
Given my Riot account is linked, **When** I click "Update recommendations", **Then** the system fetches my last 20 eligible games again, replaces my previous recommendations and shows the new ones within 20 seconds.

US-08: Monitor the Riot API usage
As an **Administrator**, I want to see a dashboard of the Riot API usage so that I stay within Riot's rate limits and detect problems early.
Acceptance criteria 1
Given I am on the Admin Dashboard, **When** one minute passes, **Then** I see the requests per minute versus the allowed limit, the error rate and the requests queued by the rate limiter, refreshed at least every minute.
Acceptance criteria 2
Given the usage reaches 80% of the limit, **When** the dashboard refreshes, **Then** I see a warning on the dashboard.

US-09: Update the static data after a patch
As an **Administrator**, I want to update the static game data when a new patch is released so that the recommendations use the current patch.
Acceptance criteria 1
Given Data Dragon reports a new patch version, **When** I trigger the update, **Then** champions, items, runes and images are replaced, the stored patch version changes and the existing tag mappings are kept.
Acceptance criteria 2
Given the new patch adds or removes game entities, **When** the update finishes, **Then** the removed ones are marked inactive and the new ones are listed without tags for my review.

US-10: Edit tags and tag mappings
As an **Administrator**, I want to edit tags and tag mappings so that the recommendations stay accurate after each patch.
Acceptance criteria 1
Given I am logged in as an administrator, **When** I change a weight (between 0 and 1) or a tag description, **Then** the change is saved and used in the next recommendation.
Acceptance criteria 2
Given I am logged in as a player, **When** I try to edit a tag or a mapping, **Then** the action is denied.

3. User Story Mapping

The table lists each user story, what it accomplishes, the functional requirements it covers and the release in which it is developed.

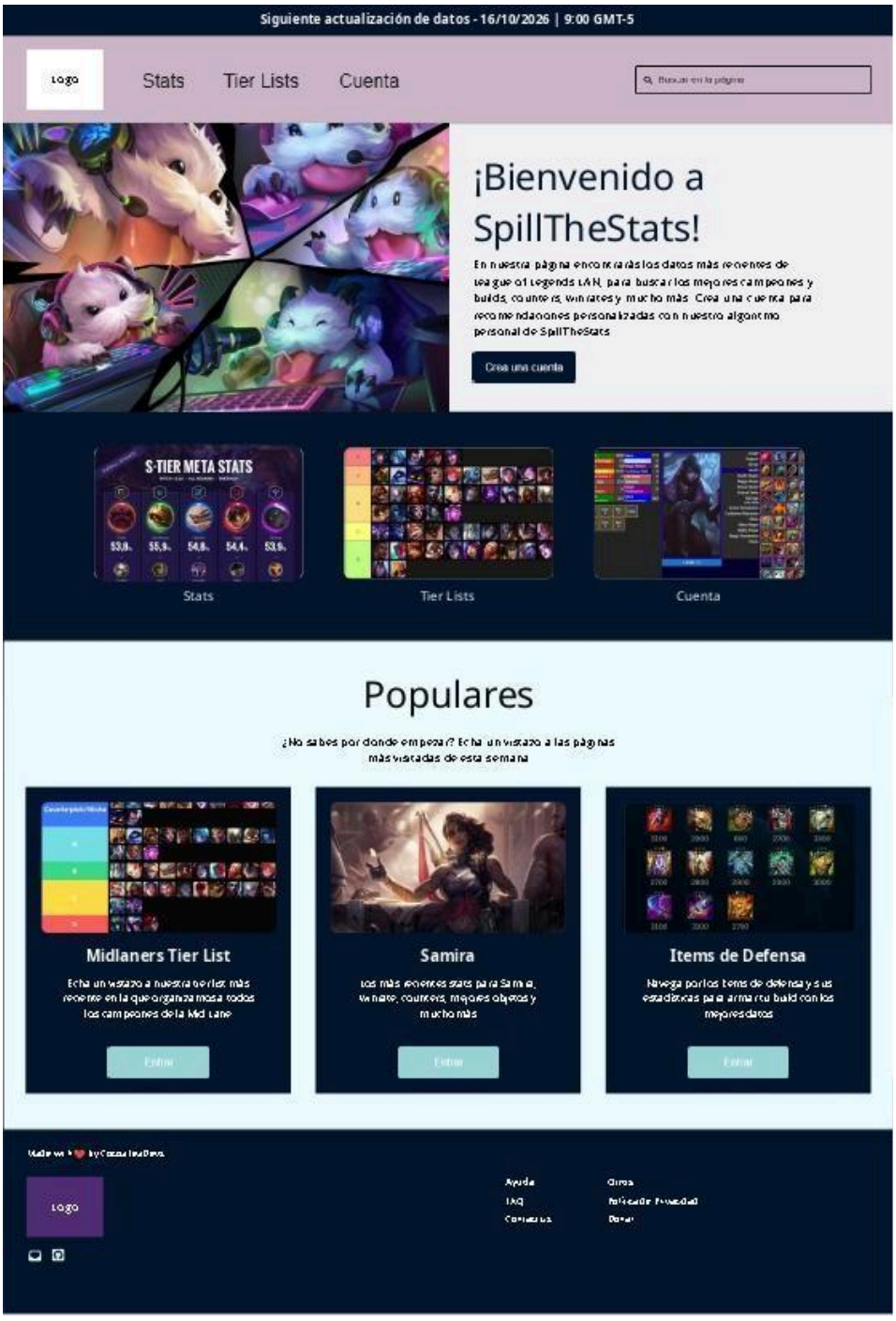
Story	What it accomplishes	Requirements
Release 1: MVP (Minimum Viable Product)		
US-01	A player can register and log in.	FR-01, FR-02
US-02	A player links the Riot ID so that the system can read their games.	FR-04, FR-05
US-03	A player sees recommended champions and a rune path, with a message when the Riot limit is near.	FR-06, FR-07, FR-08, FR-10, FR-11, FR-18

Story	What it accomplishes	Requirements
US-04	A player is warned when the result may not be conclusive, and still sees it.	FR-09
US-05	A player deletes their account and all their data.	FR-03
US-07	A player refreshes the recommendations after new games.	FR-13
Release 2: Items and administration		
US-06	A player sees an ordered item build with situational items.	FR-12
US-08	An administrator monitors the Riot API usage and sees the 80% warning.	FR-16, FR-17
US-09	An administrator updates the static data after a patch.	FR-14
US-10	An administrator edits tags and tag mappings.	FR-15
To be defined (not scheduled)		
TBD	Premium plan: deeper analysis of the matches and payment or subscription (section 1.4).	-

Releases of the project

- **Release 1: MVP.** *Goal:* allow a player to register, link their Riot account, see champion and rune recommendations (with a warning when the result may not be conclusive), update them after new games and delete their account. *Stories:* US-01 to US-05 and US-07. *Definition of Done:* the player completes the flow from registration to the recommendations and can update them without errors, and can delete the account. In this release the static game data is preloaded by the developers so that the app works.
- **Release 2: Items and administration.** *Goal:* complete the base experience for players and give administrators control of the system. *Stories:* US-06 and US-08 to US-10. *Definition of Done:* players see the item build; administrators can check the Riot API usage, update the static data after a patch and edit the tag mappings.
- **Next releases (to be defined).** Premium features, once the team defines them.

4. Mockups



This Mockup is a simple way to define what we want to achieve with the web app, it might change in the future and it does not reflect everything that will be put on the final deploy of the app. Its text is a placeholder text in Spanish; the final interface uses English (NFR-13).

5. References

- This document was made with the help of Claude's AI; everything was reviewed to ensure accuracy.
- Riot Games. Riot Developer Portal, API documentation, Data Dragon (static game data) and API policies. https://developer.riotgames.com
- plan.io. User Story Mapping. https://plan.io/blog/user-story-mapping/
- itero.gg. Inspiration for the product vision and general goals.
- Software Engineering II course material: how to write good requirements (templates for functional requirements, non-functional requirements and user stories).